# Quantum-Resistant Key Exchange API: Secure Post-Quantum Communication with Kyber KEM

---

## Table of Contents



---

## Overview

The **Quantum-Resistant Key Exchange API** leverages **Kyber Key Encapsulation Mechanism (KEM)**, a leading **NIST post-quantum cryptography standard candidate**, to enable secure key exchange in the era of quantum computing. This API allows developers to

implement **lattice-based encryption**, ensuring **forward secrecy** and resistance against **quantum-enabled attacks**.

**Target Audience:** Security engineers, backend developers, cloud architects, and developers integrating post-quantum security into APIs, SDKs, or enterprise applications.

---

# Technical Background

## What is Kyber KEM?

Kyber is a **lattice-based post-quantum cryptographic algorithm** that relies on **Module Learning with Errors (Module-LWE)** problem. Unlike classical asymmetric encryption methods like RSA or ECC, Kyber remains secure against quantum adversaries, including **Shor's algorithm**.

### Key Properties:

- **Quantum-resistant:** Secure against attacks from both classical and quantum computers.
- **Efficient:** Lightweight and fast key encapsulation suitable for cloud-native architectures.
- **Forward Secrecy:** Each session uses a unique ephemeral key, preventing compromise of past sessions.
- **Compact Key Size:** Optimized for modern protocols like TLS 1.3 and IoT devices.

### Use Cases:

- End-to-end encrypted messaging
- TLS and HTTPS upgrades for post-quantum safety
- Hybrid cryptography in cloud-native applications
- Blockchain and secure ledger communications

---

# Authentication

All requests require **API Key** or **OAuth 2.0 Bearer Tokens**.

## Header Example:

Authorization: Bearer <ACCESS_TOKEN>

Content-Type: application/json

**Note:** The API enforces **rate limiting** for security: 500 requests/min per key.

---

# API Endpoints

## 1. Generate Key Pair

**Endpoint:** /keys/generate
**Method:** POST

**Description:** Generates a **Kyber KEM public/private key pair** for secure key exchange.

### Request Body:


```
{
  "security_level": "Kyber512|Kyber768|Kyber1024",
  "key_usage": "session|long-term"
}
```


### Response:


```
{
  "public_key": "<BASE64_ENCODED_PUBLIC_KEY>",
  "private_key": "<BASE64_ENCODED_PRIVATE_KEY>",
  "timestamp": "2025-10-16T00:00:00Z",
  "security_level": "Kyber512"
}
```


### Notes:

- Kyber512 for low-latency devices
- Kyber1024 for high-security applications

---

## 2. Encapsulate Key

**Endpoint:** /keys/encapsulate
**Method:** POST

**Description:** Encapsulates a symmetric key using the recipient's **Kyber public key**.

### Request Body:

```
{
  "public_key": "<BASE64_ENCODED_PUBLIC_KEY>",
  "session_id": "abcd1234"
}
```

### Response:

```
{
  "ciphertext": "<BASE64_ENCODED_CIPHERTEXT>",
  "shared_secret": "<BASE64_ENCODED_SHARED_SECRET>",
  "session_id": "abcd1234",
  "timestamp": "2025-10-16T00:01:00Z"
}
```

### Remarks:

- The shared_secret is derived independently by both parties.
- Ciphertext is safe to transmit over public channels.

---

## 3. Decapsulate Key

**Endpoint:** /keys/decapsulate
**Method:** POST

**Description:** Decapsulates the received ciphertext to retrieve the **shared secret** using the private key.

**Request Body:**

```
{
  "private_key": "<BASE64_ENCODED_PRIVATE_KEY>",
  "ciphertext": "<BASE64_ENCODED_CIPHERTEXT>"
}
```

**Response:**

```
{
  "shared_secret": "<BASE64_ENCODED_SHARED_SECRET>",
  "timestamp": "2025-10-16T00:02:00Z"
}
```

---

## Request & Response Structure

All requests use **JSON**.
Responses are **JSON-formatted**, base64-encoded cryptographic material.

| Field | Type | Description |
|---|---|---|
| public_key | string | Kyber public key (Base64) |
| private_key | string | Kyber private key (Base64, confidential) |
| shared_secret | string | Derived session key (Base64) |
| ciphertext | string | Encapsulated key (Base64) |
| security_level | string | Kyber512, Kyber768, Kyber1024 |
| timestamp | string | UTC ISO 8601 timestamp |
| session_id | string | Optional ID for session tracking |

---

# Usage Examples

## Python Example

```
import requests

import base64


API_URL = "https://api.quantum-secure.com/v1/keys/generate"

API_KEY = "<YOUR_API_KEY>"


headers = {

    "Authorization": f"Bearer {API_KEY}",

    "Content-Type": "application/json"

}


payload = {

    "security_level": "Kyber1024",

    "key_usage": "session"

}


response = requests.post(API_URL, json=payload, headers=headers)

data = response.json()


public_key = data['public_key']

private_key = data['private_key']


print("Public Key:", public_key)
```

```
print("Private Key:", private_key)
```

---

## Security Considerations

- **Zero-Knowledge Key Exchange:** Private keys never transmitted over the network.
- **Post-Quantum Safety:** Resistant to attacks from quantum computers.
- **Ephemeral Session Keys:** Use short-lived keys for session-based communication.
- **Key Rotation:** Regularly rotate long-term keys to prevent exposure.
- **TLS Integration:** Recommended to implement Kyber KEM in TLS hybrid mode.

---

## Best Practices

1. Always make sure the **base64 encoded** keys are valid.
2. Do a hybrid **encryption using Kyber KEM + AES-GCM.**
3. Keep Track of Your API Usage And Implement Rate **Limiting** To Prevent DoS Attacks.
4. HSM: Store your private key straight to **Hardware Security Module.**
5. Carrying out a **cryptographic compliance** audit to ensure that you are NIST PQC compliant.

---

# FAQ

**Q1:** Can Kyber KEM replace RSA in my existing infrastructure?
**A1:** Yes, especially in hybrid mode, to maintain backward compatibility while ensuring quantum resistance.

**Q2:** Which security level should I choose?
**A2:** Kyber512 for IoT/low-latency, Kyber1024 for high-security enterprise or cloud applications.

**Q3:** How does Kyber resist quantum attacks?
**A3:** It relies on the hardness of **lattice problems**, which remain secure against Shor’s and Grover’s algorithms.

---

## References & Further Reading


- Kyber KEM Specification - PQCrypto
- NIST Post-Quantum Cryptography Project
- Hybrid Cryptography with Kyber & AES-GCM
- Bernstein, J., et al. *“Post-Quantum Cryptography: Lattice-Based Key Exchange”*